

SNU-GPT: A Uni-Wide Assistant

Jia Khot
B.Tech CSE
2310110491
jk637@snu.edu.in

Lalit Maurya
B.Tech CSE
2310110164
lm742@snu.edu.in

Rachit Kumar
B.Tech CSE
2310110234
rk551@snu.edu.in

Abstract—SNU-GPT is a retrieval-augmented generation (RAG) agent designed to assist university students without relying on LLM-based metadata filtering or fine-tuning. Document collections are pre-clustered too coarsely for conventional pruning methods, so we use a voting-based mechanism to dynamically select clusters. This enables efficient retrieval and response generation while reducing computational overhead, offering a lightweight, scalable solution for student-focused knowledge assistance.

I. INTRODUCTION

Retrieval-augmented generation (RAG) combines large language models (LLMs) with external knowledge sources, typically retrieving relevant documents and generating responses from them. Many RAG systems rely on LLMs for metadata filtering or fine-grained document categorization, which is computationally expensive. In university settings, document collections are often pre-clustered too coarsely for conventional pruning or leader election methods, making traditional filtering approaches inefficient.

We propose SNU-GPT, a RAG agent that replaces costly LLM-based filtering with a voting-based mechanism to dynamically select among pre-categorized clusters. This approach enables efficient retrieval and response generation, providing a lightweight and scalable solution for student-focused knowledge assistance.

II. LITERATURE REVIEW

- 1) IDF (Inverse Document Frequency) : Proposed by Karen Spärck Jones (1972) [1], IDF down-weights very common terms and gives more weight to rarer terms, reflecting their higher informational content in retrieval tasks. Rare terms are more discriminative and thus more valuable for distinguishing relevant documents.
- 2) Probabilistic Term Weighting: Robertson & Spärck Jones (1976) [2] formalized the intuition behind IDF into a probabilistic framework. They showed that the weight of a term in retrieval should depend on its probability of occurrence in relevant versus non-relevant documents. Terms that occur frequently in relevant documents but rarely in the general corpus receive higher weights, while ubiquitous terms are down-weighted. This framework underlies modern probabilistic retrieval models and informs ranking functions like BM25.
- 3) Retrieval-Augmented Generation (RAG): Lewis et al. (2020) [3] introduced RAG, which combines a dense

retriever (non-parametric memory) with a seq2seq generator (parametric memory). This enables language models to incorporate up-to-date external knowledge, improving factual accuracy, interpretability, and adaptability.

- 4) Document Chunking with Overlap: Large documents are split into overlapping chunks to preserve context and reduce information loss at boundaries. Overlapping ensures that key concepts are captured in multiple chunks, improving retrieval coverage and semantic coherence.
- 5) LLM-Based Metadata Filtering: Large language models can extract structured metadata from user queries to guide retrieval. In the AWS Bedrock “intelligent metadata filtering” approach, the LLM identifies attributes (e.g., dates or categories) and outputs them in a structured format, which is then used as filters for the retriever. This improves precision by narrowing search results, but unlike SNU-GPT’s IDF-weighted voting over collections, it relies on LLM interpretation, adding computational overhead. [4]

III. PROPOSED METHODOLOGY

A. Database Seeding

A key step in our RAG pipeline is constructing a structured and searchable database of documents. To achieve this, we implement a modular seeding framework that parses raw documents, generates embeddings, and populates MongoDB collections with structured chunks.

At the core of this framework is an abstract `Seeder` class, which standardizes the process of reading documents, generating embeddings using a pre-trained `SentenceTransformer`, and inserting processed chunks into the database. Each subclass of `Seeder` specifies how documents are split into chunks and which metadata fields are included.

For example, the `FacultySeeder` processes faculty documents stored in a directory, extracting the faculty member’s content, school, and department. Schools and departments are mapped to predefined enumerations (`SchoolEnum` and `DepartmentEnum`) to maintain consistency across the database. Each document is encoded into a fixed-length vector embedding using the `SentenceTransformer` model, which is stored alongside the document content and metadata

in a `FacultyChunk`. Once all chunks are generated, they are inserted into the corresponding MongoDB collection.

This design is generalized across multiple collections (e.g., students, academics, menu), allowing consistent storage and retrieval. By organizing documents into typed chunks with embeddings and structured metadata, the seeding process ensures that downstream retrieval operations in the RAG pipeline can be performed efficiently and accurately.

When a document exceeds a predefined size, it is split into smaller chunks to ensure efficient embedding and retrieval. Each chunk is limited to a maximum character count, and to preserve contextual continuity, consecutive chunks overlap by one-third of their content. This overlap ensures that important information spanning chunk boundaries is not lost, allowing the retrieval system to maintain semantic coherence while working with large documents.

B. Term based Voting

Our goal is to completely eliminate the use of an LLM in the metadata filtering phase of the RAG pipeline. To achieve this, we build a vocabulary for each document collection. As we parse through the documents, all words are lemmatized and passed through a part-of-speech (POS) tagger. Only nouns and verbs are retained to prevent the vocabulary from becoming excessively large. Each retained term is stored in a dictionary along with its document frequency, which is incremented whenever the term appears again.

After processing the entire collection, we calculate the inverse document frequency (IDF) for each term, providing a measure of its importance within the collection. The IDF scores are then normalized to a $[0, 1]$ range using the maximum IDF value as reference. Additionally, we track the frequency of each term across all collections. When selecting which collection to query, each term casts a vote for the collection where it appears most frequently, weighted by its normalized IDF score. The rationale for using collection frequency is that a term appearing more often in a given collection is more likely to indicate that the collection is relevant to queries containing that term. This voting-based approach ensures that more informative and representative terms have greater influence in guiding the retrieval process.

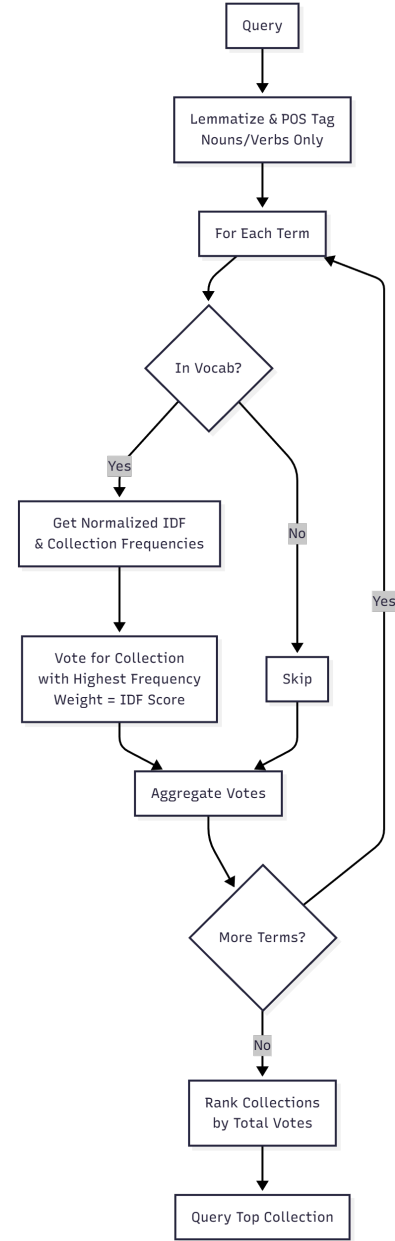


Fig. 1. Voting pipeline for a sample query

C. Generation after Retrieval

Once relevant documents are retrieved from the selected collections, SNU-GPT generates a response by combining context from the vector search with the user’s prompt and conversation history. Recent interactions are included to maintain coherence, while the system provides a concise and contextually grounded reply. The response is generated using the OpenAI GPT model, with context, history, and prompt passed as structured input to ensure that the output is relevant, accurate, and appropriate for university users. Errors during generation are caught and handled gracefully, allowing the system to provide fallback messages when necessary.

TABLE I
PRECISION@K SCORES FOR SNU-GPT RETRIEVAL

Metric	Score
P@1	0.9333
P@2	0.7333
P@3	0.5778
P@4	0.5000
P@5	0.4933

D. E-Mails and Weekly Menu

To provide users with up-to-date information, SNU-GPT integrates sources such as university emails and the weekly menu. Relevant emails-such as announcements, circulars, and notifications-are scraped and stored as structured document chunks with metadata like source, date, and type, enabling efficient retrieval. Due to budget constraints, the email collection is currently static; running a dedicated email client to continuously listen for incoming messages is not feasible.

The weekly menu is scraped from DigiCampus and stored as a single chunk representing the entire week’s offerings. This allows users to query meal information in a centralized format. Despite being static for now, these dynamic sources complement the static academic and faculty knowledge in the main database, ensuring users can access timely and relevant information.

IV. PRECISION EVALUATION

We evaluated SNU-GPT’s retrieval performance using Precision@K (P@K) to measure the relevance of retrieved documents.

The high P@1 score shows that the top-ranked document is almost always highly relevant, while the relevance decreases for lower-ranked documents, indicating room for improvement in ranking beyond the top result.

V. CONCLUSIONS AND LIMITATIONS

SNU-GPT demonstrates that a lightweight RAG agent can assist university students effectively without relying on LLM-based metadata filtering or fine-tuning. By combining pre-categorized document clusters with a voting-based retrieval mechanism, it achieves efficient and contextually relevant responses while minimizing computational overhead. Integration of dynamic sources such as emails and the weekly menu further allows users to access timely information, complementing the static academic and faculty knowledge stored in the database.

Limitations:

- 1) **Limited knowledge base and lack of formatting:** Current documents are largely unstructured, which can reduce the clarity and utility of responses. This can be addressed through manual curation and formatting of the knowledge sources.
- 2) **Non-timestamped collections:** All documents are treated equally regardless of age, so older and newer

information carry the same weight during retrieval, potentially affecting relevance.

- 3) **Token inefficiency due to overlapping chunks:** The current one-third overlap between chunks ensures context continuity but increases token usage. This could be improved by reducing overlap or switching to token-based chunking rather than character-based chunking.
- 4) **Evaluation framework:** A proper evaluation methodology to measure retrieval quality, response accuracy, and user satisfaction is not yet implemented.
- 5) **Static email and menu scrapers:** The dynamic data sources are currently static due to budget constraints; emails and weekly menus are not continuously updated. This is easily fixable with a dedicated server infrastructure to run scrapers in real time.

REFERENCES

- [1] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972, [Online]. Available: <https://www.cl.cam.ac.uk/archive/ksj21/ksjdigipapers/jdoc72.pdf>
- [2] S. E. Robertson and K. Spärck Jones, “Relevance Weighting of Search Terms,” *Journal of the American Society for Information Science*, vol. 27, no. 3, pp. 129–146, 1976, [Online]. Available: https://econpapers.repec.org/article/blajamest/v_3a27_3ay_3a1976_3ai_3a3_3ap_3a129-146.htm?utm_source=chatgpt.com
- [3] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020, pp. 9459–9474. [Online]. Available: <https://papers.nips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>
- [4] Amazon Web Services, “Streamline RAG Applications with Intelligent Metadata Filtering Using Amazon Bedrock.” Nov. 2024.